커널이 파일을 다루는 방식 → 스크립트 자동화

오전: fd 테이블 · struct file · dentry · inode · fork 그림을 세운다
오후: 그 그림 위에서 collect.sh 하나를 키워 systemd 서비스로 올린다

오늘의 도착점

Jetson을 켜기만 하면 로그 수집 스크립트가 자동으로 뜨고, SSH를 끊어도 죽지 않으며, 죽으면 스스로 다시 뜬다 — 그리고 그 한 줄 한 줄이 커널 안의 어떤 구조를 움직이는지 그림으로 설명할 수 있다.

▶ 도구 모음 (index)

📋 Day 3 워크시트 (worksheet.html)

기본 주소 https://devops0906.github.io/AI_CAMPUS/os-day3/anim/

어제까지는 겉모습, 오늘은 속

Day 1~2에서 본 트리·경로·마운트·권한·장치 파일은 전부 '이름'의 세계였습니다. 오늘은 이름이 사라진 뒤 커널 안에서 무엇이 움직이는지를 봅니다.

Day 1~2 · 겉모습

- / 에서 뻗는 트리
- 절대경로 · 상대경로
- 마운트 — 가지 접붙이기
- rwx 권한 — 세 사람과 문
- /dev · /sys — 장치도 파일

→

Day 3 오전 · 커널 내부

- open() → fd 테이블·struct file·dentry·inode
- read()/write() → 페이지 캐시·dirty
- close()/rm → 참조 카운트
- fork()/dup2() → fd 복사·파일 공유

→

Day 3 오후 · 자동화

- ./collect.sh → execve·#!
- fork·exec·wait·$?
- 변수·조건·파이프
- SIGHUP·cron·systemd

오후의 모든 현상(권한 오류 · cd가 안 먹음 · 변수 소실 · 파이프 · SIGHUP)은 오전 그림으로 되돌아가 설명됩니다.

DAY 3 · 오늘의 지도

8세션 — 오전은 그림, 오후는 그 그림 위의 스크립트

S1 #1 #4

open() · close()

fd · struct file · dentry · inode

S2 #2 #3

read() · write()

페이지 캐시 · dirty · writeback

S3 #5

fork() · dup2()

fd 복사 · struct file 공유

S4 #11 #12

실행 · 프로세스

#! · execve · fork·exec·wait

S5 #13 #15

변수 · 분기

./ source export · $? · && ||

S6 #14

파이프

버퍼 · 동시 실행 · SIGPIPE

S7 #16 #17

백그라운드 · cron

SIGHUP · nohup · 1분 루프

S8 #18

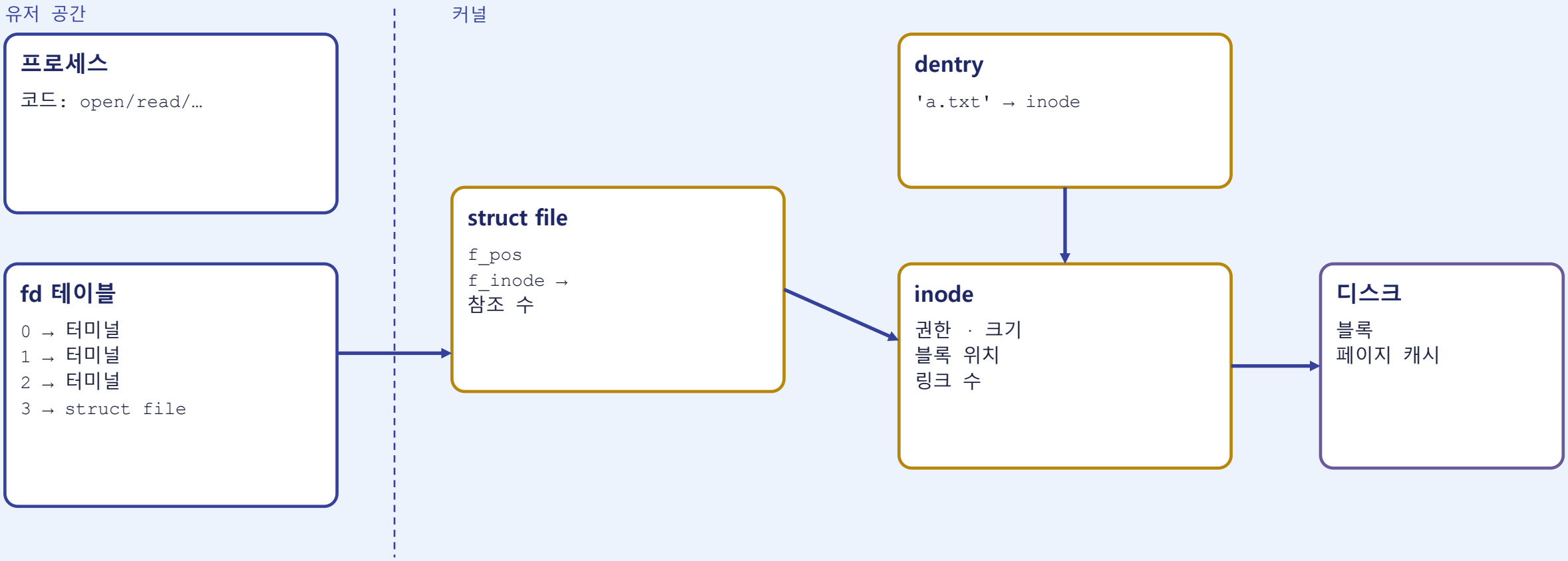
systemd (도착점)

부팅 · Restart · unit 10줄

색칠된 세 칸이 오전(커널 그림). 숫자는 도구(애니메이션) 번호 — 워크시트의 도구 ▶ 버튼과 같습니다.

오늘 하루 재사용할 그림 하나

오전 세 세션이 이 다섯 상자를 세우고, 오후 다섯 세션이 이 위에서 돕니다. 칠판에 고정으로 그려 두세요.



fork()는 왼쪽 두 상자를 복사하고, 가운데 struct file은 공유한다 — 오후의 리다이렉션·파이프·스크립트는 전부 이 한 문장의 응용

S1

open() 한 번에 커널 안에서 벌어지는 일

이 세션의 도착점

fd가 배열 인덱스이고, 파일은 이름이 아니라 inode라는 것을 /proc/<PID>/fd 로 눈으로 확인한다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"fd는 그냥 배열 인덱스이고, 파일은 이름이 아니라 inode다."

open("/home/user/a.txt") — 아홉 단계

도구 #1 정상 open 시나리오와 같은 순서입니다. 경로는 dentry에서 끝나고, 그 뒤는 전부 inode입니다.

1	시스템 콜 경계를 넘어 커널로
2~5	dentry 캐시를 / → home → user → a.txt 순서로 짚음
5	마지막 dentry가 inode #8812를 가리킴 — 이름은 여기까지
6	inode의 권한 비트 검사 (셸이 아니라 커널)
7	struct file 생성: f_pos=0, inode 가리킴, 참조 수 1
8	fd 테이블 빈 슬롯: 0·1·2 사용 중 → 3
9	3번 슬롯 → struct file, 숫자 3 반환

없는 파일이면?

5단계에서 멈춤. 음수 dentry가 남고 struct file은 만들지 않음.

→ -1, errno = ENOENT

권한이 없으면?

6단계에서 멈춤. 이름 찾기는 성공했지만 inode 검사 실패.

→ -1, errno = EACCES ('Permission denied')

rm 했는데 df가 안 줄어드는 이유

이름표(dentry) · 열린 파일(struct file) · 본체(inode·블록)는 각각 따로 사라집니다 — 도구 #4의 세 시나리오.

close()만

fd 슬롯 비움 → 참조 수 0 → struct file 해제

inode·블록은 그대로.

close는 '지우기'가 아니라 '손 떼기'

열린 채로 rm

이름표 제거, 링크 수 0

그러나 struct file이 inode를 잡고 있음

→ 블록 해제 안 됨, df 그대로

→ /proc/PID/fd 에 (deleted)

마지막 close 때 비로소 해제

안 열린 파일 rm

링크 수 0 + 열린 파일 0

→ 즉시 inode·블록 해제

(내용은 덮어쓰기 전까지 남아 있을 수 있음)

현장: 커다란 로그 파일을 rm 했는데 여유 공간이 안 늘면, 그 파일을 열고 있는 프로세스를 찾아 close(재시작)해야 한다.

S1 실습 — 눈에 보이는 산출물 만들기

1. python3 -c "f=open('a.txt','w'); import os; print(f.fileno()); input()" 실행 → 3 출력 확인
2. 다른 터미널: ls -l /proc/<PID>/fd → 3번 슬롯이 a.txt 인 것 확인
3. 그대로 rm a.txt → 다시 ls -l /proc/<PID>/fd → (deleted) 확인
4. 프로세스 종료(Enter) → 항목 사라짐
5. 워크시트 S1 6문항 → 제출 화면 만들기 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#1 open() — 경로 탐색에서 fd까지

#4 close()·rm — 참조 카운트

📋 Day 3 워크시트 — S1 제출 화면

확인 문구 「왼쪽 화면에서 실패한 건 셸이 막은 건가요, 커널이 막은 건가요?」→ 전원 "커널"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S2

read()·write() — 디스크는 언제 실제로 움직이나

이 세션의 도착점

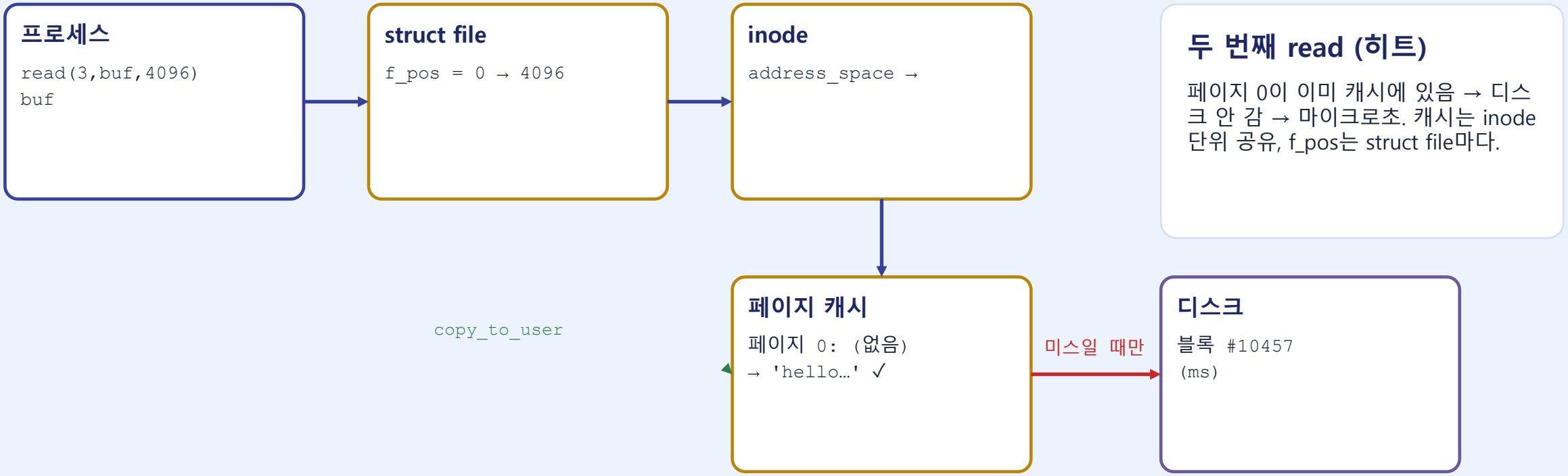
read는 두 번째부터 빠르고, write는 아직 디스크에 없다 — time cat 세 번으로 증명한다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

“read는 두 번째부터 빠르고, write는 아직 디스크에 없다.”

read(3, buf, 4096) — 캐시 미스 vs 히트

S1 그림의 inode 오른쪽에 페이지 캐시와 디스크 상자를 붙입니다. 디스크는 미스일 때만 움직입니다.



Jetson에서 같은 모델 파일을 두 번째 로드할 때 빠른 이유가 이 그림입니다.

write()는 페이지 캐시에만 쓰고 돌아온다

도구 #3의 세 시나리오. 디스크에 닿는 시점을 정하는 것은 writeback 스레드 또는 fsync입니다.

방치

write → 페이지 dirty → 즉시 반환
약 30초 뒤 writeback 스레드가 flush
→ 그제야 디스크가 'new'

fsync

write → dirty → fsync(3)
프로세스가 잠들고 블록 쓰기 완료 후 반환
→ 반환 전까지 디스크 보장

전원 차단

write 반환 = 3 (성공)
→ 전원 차단 → RAM 소멸 → dirty 페이지 소멸
→ 재부팅 후 'hello'

현장 규칙

Jetson SD카드를 뽑거나 전원을 끊기 전에 sync. 'sync 없이 전원 뽑지 마라'는 미신이 아니라 이 그림입니다. (저널링이 지키는 것은 메타데이터이지 방금 쓴 데이터가 아님 — Day 4 심화)

S2 실습 — 눈에 보이는 산출물 만들기

1. dd if=/dev/urandom of=big bs=1M count=200 으로 200 MB 파일 생성
2. time cat big > /dev/null 두 번 → real 시간 비교 (느림 → 빠름)
3. free -h 의 buff/cache 값 관찰
4. echo 3 | sudo tee /proc/sys/vm/drop_caches → time cat 다시 (느려짐)
5. 워크시트 S2 7문항 → 제출 화면 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#2 read() — 페이지 캐시

#3 write() — dirty·writeback

📋 **Day 3 워크시트 — S2 제출 화면**

확인 문구 「Jetson 전원을 그냥 뽑으면 방금 저장한 파일이 사라질 수 있는 이유는?」→ "dirty 페이지가 아직 안 내려가서"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S3

fork()·dup2() — 프로세스가 복제될 때 fd는 어떻게 되나

이 세션의 도착점

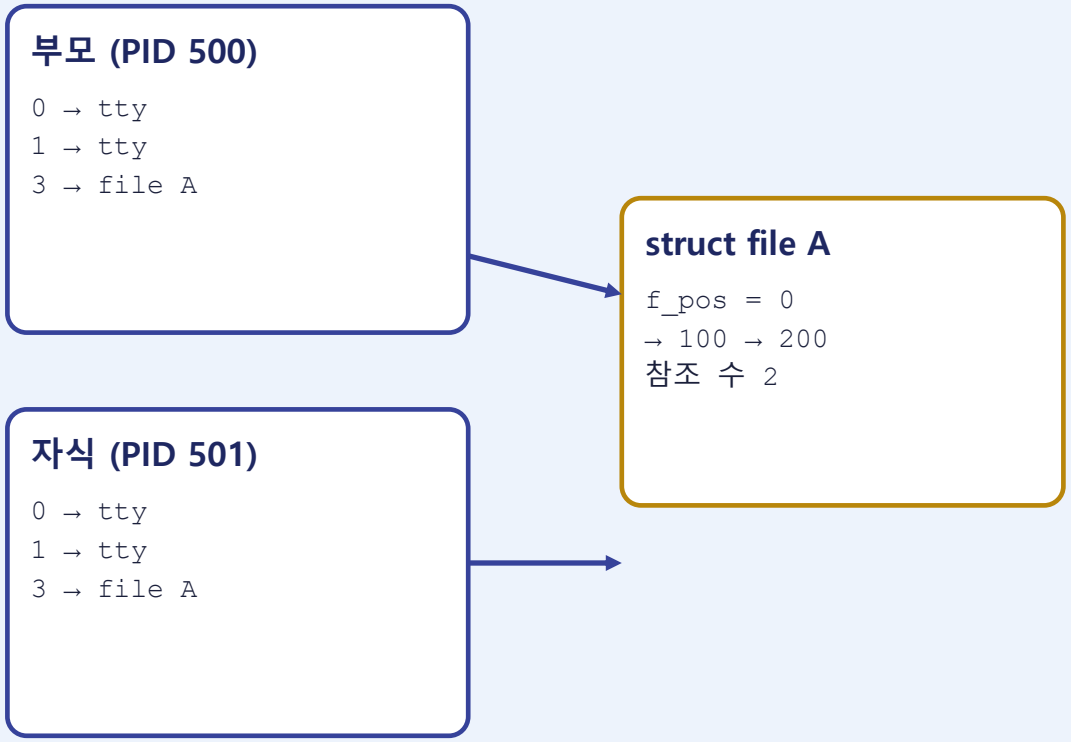
fd 테이블은 복사, struct file은 공유 — 그래서 f_pos가 같이 움직이고, 그래서 > 리다이렉션이 된다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"fd 테이블은 복사되지만 struct file은 공유된다 — 그래서 >가 된다."

복사되는 것 = 화살표, 공유되는 것 = struct file

왼쪽은 도구 #5 첫 시나리오(f_pos 공유), 오른쪽은 두 번째 시나리오(ls > out.txt)의 일곱 단계입니다.



부모가 100 B 읽으면 자식은 100부터 읽는다

ls > out.txt 의 일곱 단계

1. bash가 `ls > out.txt` 읽고 fork
2. 자식: open(out.txt) → fd 3
3. 자식: dup2(3, 1) — 1번 화살표를 파일로
4. 자식: close(3)
5. 자식: execve(/bin/ls) — fd 테이블은 유지
6. ls는 평소처럼 fd 1에 write (파일인지 모름)
7. 부모 bash의 1번은 여전히 터미널

S3 실습 — 눈에 보이는 산출물 만들기

1. ls -l /proc/$$/fd → 내 셸의 0·1·2가 /dev/pts/N 인 것 확인
2. bash 로 자식 셸 → ls -l /proc/$$/fd → 같은 pts (복사됨)
3. exec 1>out.txt → ls -l /proc/$$/fd/1 이 out.txt 인 것 확인
4. exec 1>/dev/tty 로 복귀 → cat out.txt
5. 워크시트 S3 6문항 → 제출 화면 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#5 fork·dup2 — fd 공유

📋 Day 3 워크시트 — S3 제출 화면

확인 문구 「ls > out.txt 에서 ls는 자기 출력이 파일로 가는 걸 아나요?」→ "모른다, fd 1이 바뀌었을 뿐"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

스크립트도 이 그림으로 돈다

./collect.sh 가 Permission denied	→ S1 — inode 권한 검사는 커널이 한다
스크립트 안의 cd가 터미널에 안 남음	→ S3 — 스크립트는 자식 bash, cwd는 자식 것
변수가 사라짐 / source면 남음	→ S3 — 변수는 프로세스 안에 산다
> 와 | 가 되는 이유	→ S3 — dup2로 fd 1·0의 화살표를 바꾼다
SSH 끊으면 죽음	→ S3 — 같은 세션의 자식이라 SIGHUP을 같이 받는다

오후에는 collect.sh 파일 하나를 S4에서 만들어 S8에서 서비스로 올린다. 세션마다 한 겹씩 얹는다.

S4 ./collect.sh — 스크립트가 실행되고 프로세스가 된다

이 세션의 도착점

실행 권한은 커널이 검사하고, 스크립트는 한 줄마다 fork·exec·wait를 반복하는 공장이다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"스크립트는 S3의 fork를 한 줄마다 반복하는 공장이다."

커널이 #!을 읽는 순간 — execve의 세 단계

도구 #11. 텍스트 파일이 '실행'되는 것은 커널이 첫 두 바이트를 보고 /bin/bash를 대신 띄우기 때문입니다.

1

inode의 x 비트 검사

rw-r--r-- 이면 여기서 끝 → EACCES. #!은 읽어 보지도 않음

2

첫 2바이트 읽기

'#!' 이면 스크립트. 뒤의 경로가 인터프리터

3

인터프리터 로드

/bin/bash 를 대신 exec, argv = [/bin/bash, ./run.sh]

./run.sh vs bash run.sh

bash run.sh 는 실행 대상이 /bin/bash → 커널은 /bin/bash의 x만 검사. run.sh는 r만 있으면 됨.

Permission denied 의 출처

셸이 아니라 execve()의 1단계. chmod +x 는 inode의 비트 하나를 켜는 것.

한 줄이 프로세스가 된다 — fork · exec · wait · $?

도구 #12. bash는 줄마다 이 루프를 한 바퀴 돕니다. 빌트인(cd)만 예외입니다.



자식 = bash 복사본

자식의 몸이 ls로

부모는 멈춤

종료 코드 → 커널

부모가 받음 → 다음 줄

$? — 0이 성공

ls -l → 0

ls /nonexist → 2

'0이 아니면 실패'. if · && · || 가 이 값을 본다.

cd는 왜 빌트인인가

cd를 fork로 실행하면 자식의 cwd만 바뀌고 부모는 제자리.

그래서 bash 자신이 chdir을 부른다.

스크립트 안의 cd

스크립트 자체가 터미널 bash의 자식 bash.

그 안에서 cd 해도 터미널은 제자리.

S4 실습 — 눈에 보이는 산출물 만들기

1. (5분) #10 미니 vi로 i → 입력 → Esc → :wq 순서 익히기
2. collect.sh 생성: #!/bin/bash + echo start → ./collect.sh → Permission denied
3. chmod +x collect.sh → 성공 / bash collect.sh 와 비교 / ls -l 로 x 확인
4. ls /nonexist; echo $? 를 스크립트에 넣고 실행 → 값 관찰
5. cd /tmp 를 스크립트에 넣고 실행 → 터미널에서 pwd → 제자리
6. 워크시트 S4 6문항 → 제출 화면 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#10 vi 세 모드 — 미니 vi 시뮬레이터

#11 ./run.sh — 커널이 #!을 읽는 순간

#12 fork·exec·wait·$? 루프

📋 Day 3 워크시트 — S4 제출 화면

확인 문구 「cd를 스크립트에 넣었는데 내 터미널은 왜 제자리인가요?」 → "자식 프로세스에서만 이동했다"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S4가 끝났을 때 collect.sh

collect.sh — v1

```
#!/bin/bash
# collect.sh — S4 끝의 모습
# S4에서 추가: #! 첫 줄 + echo 한 줄. chmod +x 전에는 ./collect.sh 가 Permission denied.
echo start
```

▶ scripts/collect_v1.sh

두 줄뿐이지만 커널은 이미 x 비트 검사와 #! 읽기를 했다. 이 파일 하나를 S8까지 계속 편집한다.

S5

변수의 수명과 스크립트의 판단

이 세션의 도착점

변수는 프로세스 경계를 한 방향으로만 넘고, if는 $?를 읽는다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"변수는 부모→자식 한 방향으로만 흐르고, if는 $?를 읽는다."

./ · source · export — 변수는 어디까지 살아있나

도구 #13. 변수 상자는 프로세스마다 하나. export는 execve 시점에 복사될 뿐 되돌아오지 않습니다.

./collect.sh

fork+execve → 자식 bash
자식의 상자에 LOG_DIR 생김
자식 종료 → 상자 소멸
터미널: echo $LOG_DIR → 빈 값

source collect.sh

fork 없음
터미널 bash가 직접 읽어 실행
터미널 상자에 LOG_DIR 남음
(.bashrc 를 source로 읽는 이유)

export LOG_DIR=...

execve 때 envp로 자식에 복사
자식이 LOG_DIR=/tmp 로 바꿔도
부모는 그대로
흐름은 부모 → 자식 한 방향

'스크립트에서 설정한 PATH가 왜 터미널에 안 남지?' — 자식의 상자였기 때문.

[는 명령이고, if는 $?가 0인지 본다

도구 #15. 조건문은 마법이 아니라 S4 루프의 종료 코드를 읽는 것입니다.

구문	참일 때	거짓일 때
`[ -d ~/logs ] || mkdir -p ~/logs`	있음 → [가 exit 0 → || 오른쪽 건너뜀	없음 → exit 1 → || 오른쪽 fork → mkdir
`cp a.txt b.txt && echo 복사됨`	cp 성공 → 0 → echo 실행	cp 실패 → 1 → echo는 fork조차 안 됨
`if cmd; then … fi`	cmd의 $? == 0 이면 then	0이 참인 이유: 성공은 하나, 실패 이유는 여럿

for i in 1 2 3; do date >> $LOG_DIR/test.log; done — 반복도 같은 루프를 세 번 도는 것

S5 실습 — 눈에 보이는 산출물 만들기

1. collect.sh에 LOG_DIR=~/logs 선언 → ./collect.sh 후 echo $LOG_DIR (빈 값)
2. source collect.sh 후 echo $LOG_DIR (값 있음)
3. [-d "$LOG_DIR"] || mkdir -p "$LOG_DIR" 추가 → 두 번 실행, 첫 번만 생성
4. for i in 1 2 3; do date >> $LOG_DIR/test.log; done → 3줄 확인
5. 워크시트 S5 7문항 → 제출 화면 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#13 ./ vs source vs export

#15 if · && · || — 종료 코드

📋 **Day 3 워크시트 — S5 제출 화면**

확인 문구 「[-d ~/logs]가 실패하면 $?는 몇인가요?」→ "0이 아닌 값"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S5가 끝났을 때 collect.sh

collect.sh — v2

```
#!/bin/bash
# collect.sh — S5 끝의 모습
# S5에서 추가: LOG_DIR 변수, [ -d ] || mkdir 조건, for 반복
LOG_DIR=~/logs                              # ./collect.sh 로 실행하면 이 변수는 자식 bash 안에만 존재
echo "$LOG_DIR"
[ -d "$LOG_DIR" ] || mkdir -p "$LOG_DIR"    # [ 의 $?가 0이 아닐 때만 mkdir 실행
for i in 1 2 3; do
  date >> "$LOG_DIR/test.log"
done
```

▶ scripts/collect_v2.sh

추가된 줄: LOG_DIR 변수 · [-d] || mkdir -p · for 반복. ./ 로 실행하면 LOG_DIR은 자식 bash 안에서만 산다.

S6 파이프로 로그 만들기

이 세션의 도착점

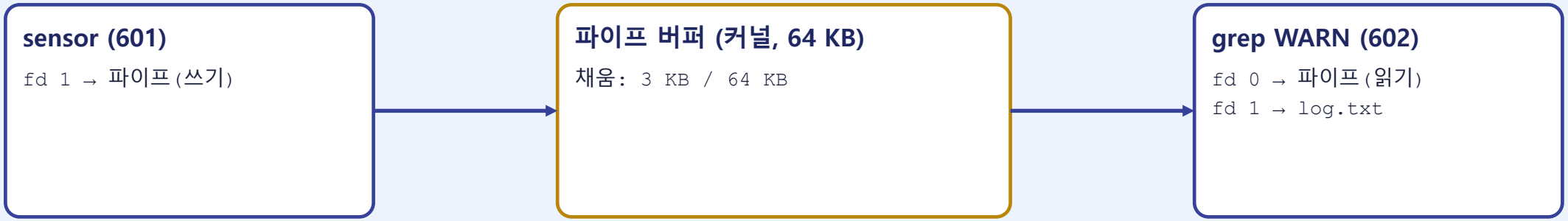
파이프는 S3의 dup2를 두 프로세스에 동시에 적용한 것이다 — collect.sh 본체가 완성된다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"파이프 양쪽 프로세스는 순서대로가 아니라 동시에 돈다."

파이프 — 커널 버퍼 하나에 두 프로세스가 꽂힌다

도구 #14. pipe()로 커널에 버퍼 하나, fork 두 번, 각자 dup2, 각자 execve. 데이터는 유저 공간을 거치지 않습니다.



정상

둘이 동시에 실행. sensor가 write → 버퍼 → grep이 read → 필터 → 파일

버퍼가 가득

write가 블록 — 커널이 sensor를 재움. 데이터는 안 버려짐. grep이 읽으면 깨어남 = 흐름 제어

오른쪽 먼저 종료

head -1 이 끝나면 읽기 끝 닫힘 → sensor의 다음 write → SIGPIPE → 조용히 종료

S6 실습 — 눈에 보이는 산출물 만들기

1. collect.sh 본체: 데이터 소스 → grep/awk 추출 → date 붙여 >> $LOG_DIR/sensor.log
2. 2>&1 로 에러도 같은 로그에 → 일부러 오류 내서 찍히는지 확인
3. ls -l /proc/$$/fd 는 안 바뀌었음을 재확인 (자식만 바뀜)
4. tail -5 sensor.log 로 타임스탬프 + 값 확인
5. 워크시트 S6 6문항 → 제출 화면 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#14 파이프 버퍼와 dup2

📋 Day 3 워크시트 — S6 제출 화면

확인 문구 「| 왼쪽이 끝나기 전에 오른쪽이 시작할 수 있나요?」 → "있다"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S6가 끝났을 때 collect.sh — 본체 완성

collect.sh — v3 (1회 실행형)

```
#!/bin/bash
# collect.sh — S6 끝의 모습 (1회 실행형)
# S6에서 추가: 파이프 본체, { } >> log 2>&1 로 stdout·stderr를 한 파일에
# 데이터 소스 [강사 확정]: 센서 미준비 시 SoC 온도로 대체, 확정되면 cat 줄만 교체
LOG_DIR=~/logs
[ -d "$LOG_DIR" ] || mkdir -p "$LOG_DIR"
{
  cat /sys/class/thermal/thermal_zone0/temp \
    | awk -v t="$(date '+%F %T')" '{printf "%s temp=%.1fC\n", t, $1/1000}'
} >> "$LOG_DIR/sensor.log" 2>&1
```

▶ scripts/collect_v3.sh

파이프 한 줄이 본체. { } >> log 2>&1 로 묶어야 cat의 오류도 같은 로그에 들어간다. 데이터 소스 줄은 [강사 확정] 후 교체.

S7

사람 없이 돌게 하기 — 백그라운드와 cron

이 세션의 도착점

터미널을 닫으면 커널이 SIGHUP을 보내고, cron은 내 터미널 환경을 모른다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"터미널을 닫으면 커널이 SIGHUP을 보내고, cron은 내 터미널 환경을 모른다."

SSH를 끊으면 왜 죽나 — SIGHUP · nohup · setsid

도구 #16. 터미널이 사라지면 커널이 그 세션 전체에 SIGHUP. 살아남는 길은 무시하거나, 세션을 떠나거나.

./collect.sh &

같은 세션·같은 제어 터미널
SSH 끊김 → 커널이 세션에 SIGHUP
기본 처리기 = 종료
재접속 후 ps → 없음

nohup ./collect.sh &

SIGHUP 처리기를 '무시'로 바꾼 뒤 exec
출력은 nohup.out
bash는 죽고 스크립트는 생존
재접속 후 ps → 있음

setsid ./collect.sh &

새 세션(SID)의 리더, 제어 터미널 없음
커널은 옛 세션에만 SIGHUP
신호 자체를 받지 않음
= systemd 서비스가 사는 방식

보낸 주체는 셸도 sshd도 아닌 커널. 시그널 하나가 세션(SID)이라는 묶음 단위로 배달된다.

cron — 매 분 깨어나 다섯 칸을 대조하는 데몬

도구 #17. crond는 기다리지 않습니다. 일치한 줄을 fork/exec 하고 다시 잠듭니다.

분	시	일	월	요일
0–59	0–23	1–31	1–12	0–7

```
* * * * *  /home/user/collect.sh     ← 매 분
0 9 * * 1  /home/user/backup.sh      ← 월요일 09:00
```

자식이 받는 환경

PATH=/usr/bin:/bin
HOME=/home/user
(터미널 변수 없음)
→ S5의 envp 복사 그림과 같음

'터미널에선 되는데 cron에선 안 됨' 1순위

python3 가 /usr/local/bin 에 있으면 cron의 PATH로는 못 찾음 → exit 127, 오류는 메일함/syslog로.

해결

절대경로(/usr/local/bin/python3) 또는 crontab 맨 위에 PATH=... 한 줄
.

S7 실습 — 눈에 보이는 산출물 만들기

1. (먼저) crontab -e 에 * * * * * /home/<user>/collect.sh 등록만 해 두기
2. collect_loop.sh(while true 루프)를 SSH에서 & 실행 → 끊고 재접속 → ps → 없음
3. nohup … & 로 재실행 → 끊고 재접속 → ps → 있음 → kill
4. sensor.log 에 1분 간격 로그 확인 → 상대경로로 바꿔 cron에서만 실패 재현 → 절대경로로 수정
5. 워크시트 S7 7문항 → 제출 화면 → 카드 캡처

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#16 SIGHUP · nohup · setsid

#17 cron — 1분 루프와 환경

📋 Day 3 워크시트 — S7 제출 화면

확인 문구 「SSH를 끊었을 때 스크립트를 죽인 건 누구?」→ "커널의 SIGHUP" / 「cron에서만 안 되면 먼저 볼 것은?」→ "환경변수·PATH"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S7가 끝났을 때 — 두 가지 형태

collect.sh — cron용 (1회형)

```
#!/bin/bash
# collect.sh — S7 cron 등록용 최종 (1회 실행형)
# S7에서 추가: PATH 명시 + 절대경로 (cron은 터미널 환경을 모른다)
# crontab -e :  * * * * * /home/user/collect.sh
PATH=/usr/local/bin:/usr/bin:/bin
LOG_DIR=/home/user/logs        # ~ 대신 절대경로, 계정명으로 교체
[ -d "$LOG_DIR" ] || mkdir -p "$LOG_DIR"
{
  cat /sys/class/thermal/thermal_zone0/temp \
    | awk -v t="$(date '+%F %T')" '{printf "%s temp=%.1fC\n", t, $1/1000}'
} >> "$LOG_DIR/sensor.log" 2>&1
```

▶ scripts/collect_v4.sh

collect_loop.sh — 백그라운드용 (루프형)

```
#!/bin/bash
# collect_loop.sh — S7 백그라운드(&·nohup) 실습 + S8 systemd 서비스용 (상시형)
# S7에서 추가: while true + sleep 5. 1회형 collect.sh와 이름을 구분
PATH=/usr/local/bin:/usr/bin:/bin
LOG_DIR=/home/user/logs
[ -d "$LOG_DIR" ] || mkdir -p "$LOG_DIR"
while true; do
  {
    cat /sys/class/thermal/thermal_zone0/temp \
      | awk -v t="$(date '+%F %T')" '{printf "%s temp=%.1fC\n", t, $1/1000}'
  } >> "$LOG_DIR/sensor.log" 2>&1
  sleep 5
done
```

▶ scripts/collect_loop.sh

이름을 나눈다: cron은 1회형 collect.sh를 1분마다 fork, & · nohup · systemd는 루프형 collect_loop.sh를 한 번 띄운다. 둘 다 PATH와 절대경로.

S8 systemd — 켜기만 하면 뜨고, 죽으면 다시 뜬다

이 세션의 도착점

오늘 만든 collect.sh를 unit 파일 10줄로 서비스에 올리고, 재부팅 후 active (running)을 확인한다.

핵심 문장 — 세션 끝에 전원이 말할 수 있어야 하는 한 줄

"오늘의 도착점을 unit 파일 10줄로 완성한다."

PID 1이 unit을 읽어 순서대로 띄우고, 죽으면 다시 fork한다

도구 #18. cron은 주기 실행, systemd는 상시 서비스. Restart=는 사실 S4의 wait + fork를 다시 하는 것입니다.

부팅

전원 → 커널 → PID 1 systemd
unit 파일 읽어 의존성 그래프
After=network.target → 네트워크 먼저
fork/exec → active (running)
터미널·세션 없음 → SIGHUP 무관

크래시

exit 1 / kill -9 → 커널이 PID 1에 알림(wait)
상태 failed → Restart=on-failure
RestartSec=5 → 5초 뒤 다시 fork
새 PID → active
journalctl -u collect 에 기록

systemctl stop

systemd가 SIGTERM 전송
사람이 멈춘 것 = 성공 종료
→ on-failure 아님 → 재시작 안 함
disable → 부팅 시에도 안 뜸

```
[Unit] Description=… After=network.target   [Service] ExecStart=/home/user/collect_loop.sh Restart=on-failure RestartSec=5
[Install] WantedBy=multi-user.target
```

S8 · 하루 종합

collect.sh 한 줄의 여행 — 오늘 배운 여덟 세션이 한 번씩 재등장

1 S8

systemd (PID 1)

collect_loop.sh를 fork/exec — 부모 = PID 1

2 S4

execve + #!

x 비트 검사 → /bin/bash 로드

3 S4·S5

한 줄마다 fork·exec·wait

$? 로 다음 줄 결정

4 S5

변수 상자

자식 bash 안에서만 존재

5 S3·S6

| 와 >>

dup2 로 fd 0·1 화살표 교체

6 S1

open(sensor.log)

dentry → inode → struct file → fd 3

7 S2

write → 페이지 캐시

dirty → writeback → 디스크

8 S7·S8

세션 없음

SIGHUP 무관, 죽으면 Restart

워크시트 S8 마지막 문항: '>> sensor.log 는 자식 fd 테이블의 몇 번 슬롯을 바꾼 것인가' — 답은 오전 그림(fd 1)으로 돌아온다.

S8 실습 — 눈에 보이는 산출물 만들기

1. crontab 항목 제거 (1회형 collect.sh는 더 이상 안 씀)
2. collect_loop.sh 로 /etc/systemd/system/collect.service 작성(ExecStart · Restart=on-failure · WantedBy) → systemctl enable --now
3. kill -9 로 죽인 뒤 systemctl status 에서 재시작 확인
4. 재부팅 → 접속 직후 systemctl status collect → active (running)
5. 워크시트 S8 7문항 → 제출 화면 → 카드 캡처 → 짝에게 1분 설명

도구 ▶ 애니메이션을 먼저 끝까지 넘긴 뒤 워크시트에 답을 적습니다

#18 systemd — 부팅·재시작·stop

📋 Day 3 워크시트 — S8 제출 화면

확인 문구 「지금 collect.sh는 누가 띄웠고, 그 안의 >>는 커널에서 무엇을 바꿨나요?」→ "systemd가 fork했고, 자식의 fd 1이 로그 파일 inode를 가리킨다"

확인 문구는 발화체 그대로 — 전원의 답이 나오면 다음 세션

S8 — 서비스로 올리는 것

collect_loop.sh (ExecStart 대상)

```
#!/bin/bash
# collect_loop.sh — S7 백그라운드(&·nohup) 실습 + S8 systemd 서비스용 (상시형)
# S7에서 추가: while true + sleep 5. 1회형 collect.sh와 이름을 구분
PATH=/usr/local/bin:/usr/bin:/bin
LOG_DIR=/home/user/logs
[ -d "$LOG_DIR" ] || mkdir -p "$LOG_DIR"
while true; do
  {
    cat /sys/class/thermal/thermal_zone0/temp \
      | awk -v t="$(date '+%F %T')" '{printf "%s temp=%.1fC\n", t, $1/1000}'
  } >> "$LOG_DIR/sensor.log" 2>&1
  sleep 5
done
```

▶ scripts/collect_loop.sh

/etc/systemd/system/collect.service

```
# /etc/systemd/system/collect.service — S8
# sudo systemctl daemon-reload && sudo systemctl enable --now collect
[Unit]
Description=Day 3 sensor log collector
After=network.target

[Service]
ExecStart=/home/user/collect_loop.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

▶ scripts/collect.service

ExecStart는 루프형. Restart=on-failure 이므로 루프가 죽으면 5초 뒤 systemd가 다시 fork한다. /home/user 는 훈련생 계정명으로 교체.

오늘 세운 그림은 지우지 않는다

오늘 한 것

- 다섯 상자(프로세스·fd 테이블·struct file·dentry·inode) + fork
- collect.sh 를 S4에서 만들어 S8에서 서비스로
- 워크시트 52문항 · 세션 카드 8장 제출
- 확인 문구 8개의 답이 전부 오전 상자 이름으로 나옴

Day 4 첫 세션 — #19 udev

센서를 꽂으면 → 커널 uevent → udevd 규칙 대조 → /dev/ttyUSB0 생성 + RUN+= 스크립트 실행.

오늘의 #6 장치 파일과 #11 스크립트 실행이 합쳐지는 지점 — Embedded I/O 로 가는 다리.

심화(#6~#10: VFS 디스패치 · 블록 매핑 · 마운트 점프 · mmap · 저널링)는 Day 4 이후.

▶ 도구 모음 (index)

📋 Day 3 워크시트 — 전체 결과 보기